Singleton Design Pattern

Creating One and Only One Object

LEARNING OUTCOMES

- **Explain** the Singleton Pattern
- **Identify** situations where Singleton is useful
- **Implement** Singleton in Java
- **Evaluate** when Singleton



PRE-CLASS ACTIVITIES

To get the most out of our class, please complete the following **before** the session.

Students should review:

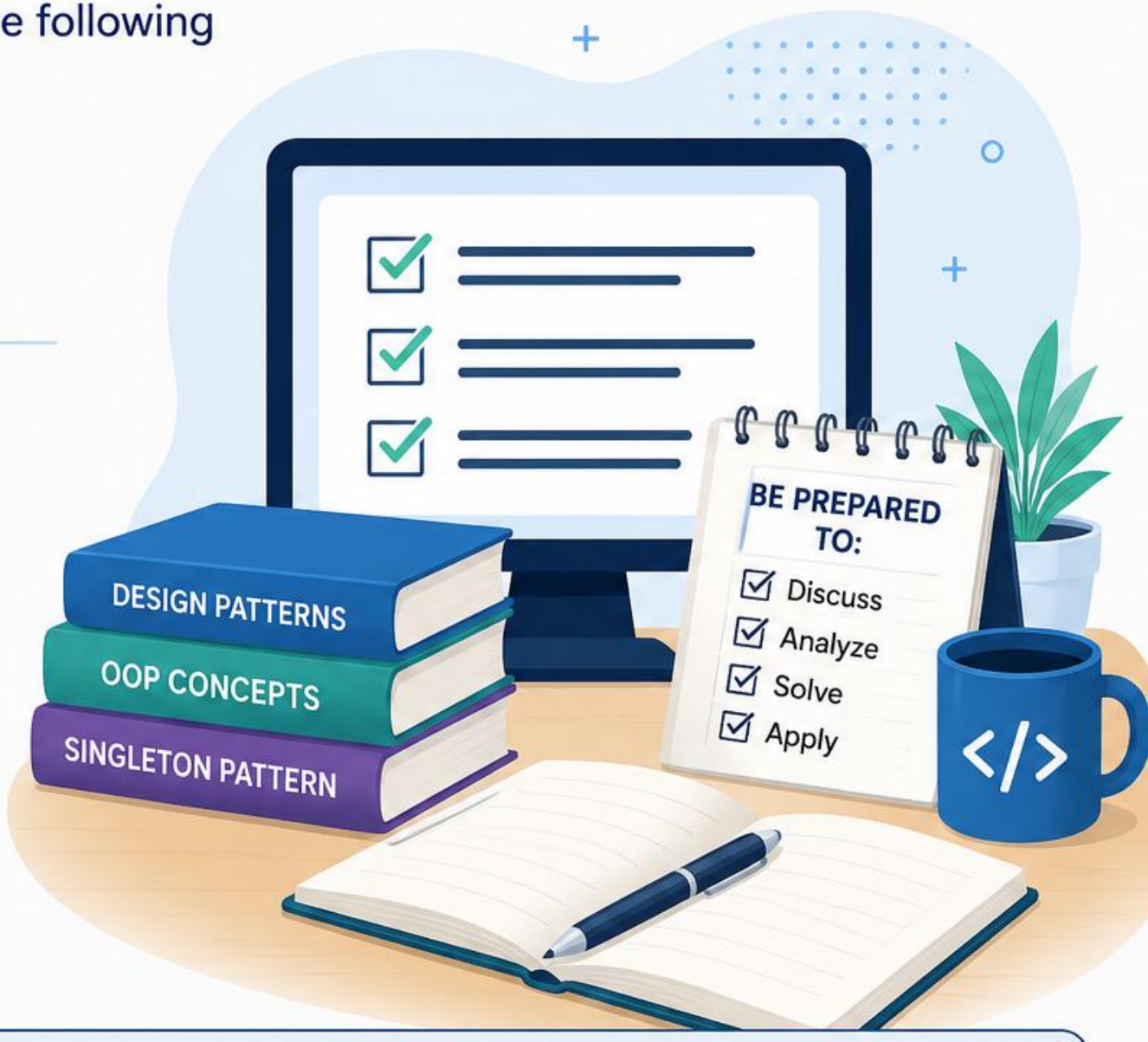
- What is a Design Pattern?
- Categories of Design Patterns

OOP Concepts:

- Classes
- Objects
- Constructors
- Encapsulation
- Static Variables and Methods

Read the Singleton notes and answer:

- Why would we want only one object?
- What happens if multiple objects are created?



QUICK QUIZ

Test your understanding of OOP concepts!

QUESTION 1

Which OOP feature prevents direct access to object data?

a. Inheritance
b. Encapsulation
c. Polymorphism
d. Abstraction

QUESTION 2

Which keyword allows a variable to belong to the class rather than an object?

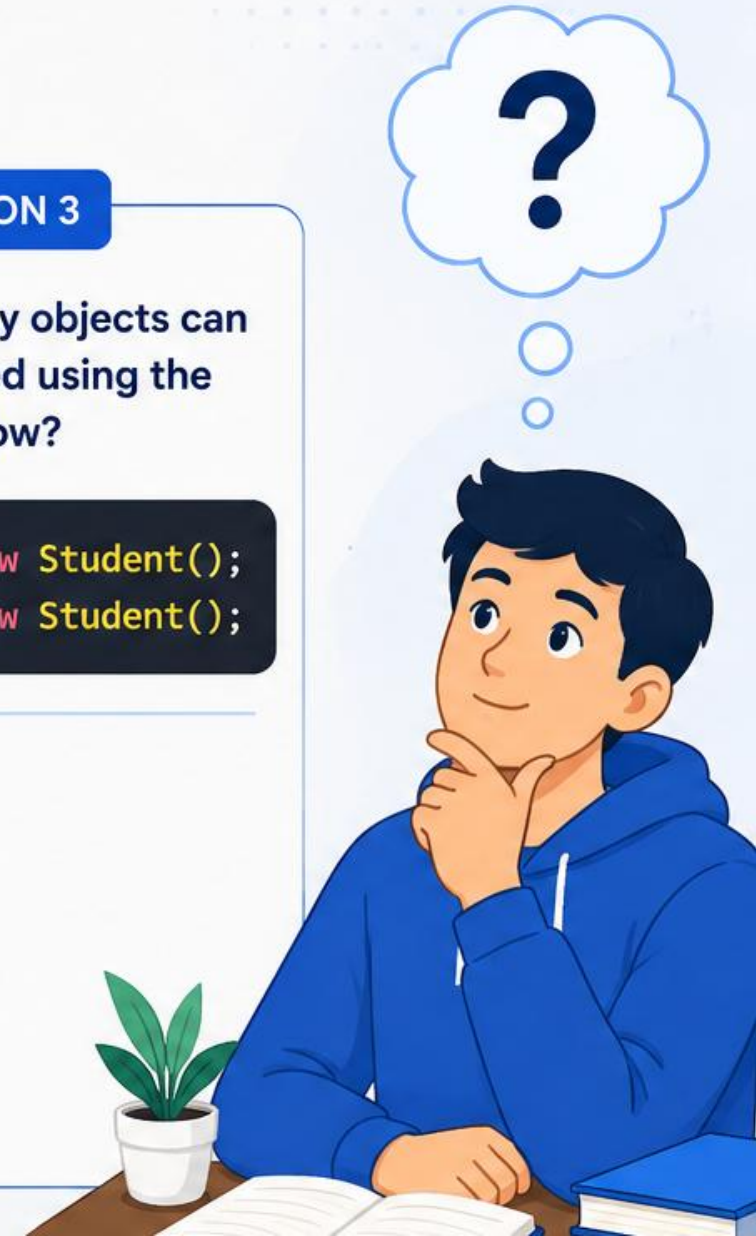
a. final
b. private
c. static
d. this

QUESTION 3

How many objects can be created using the code below?

```
Student s1 = new Student();
Student s2 = new Student();
```

a. 0
b. 1
c. 2
d. Unlimited



Which OOP feature prevents direct access to object data?

a. Inheritance

✓ b. **Encapsulation**

c. Polymorphism

d. Abstraction

REASONING

Encapsulation hides the internal data of an object and allows access only through controlled methods (getters/setters).

EXAMPLE

```
class Student {
    private String name;

    public void setName(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
}
```

Here, **name** cannot be accessed directly because it is marked as **private**.

WHY THE OTHERS ARE INCORRECT

Inheritance

Allows one class to acquire features of another class.

Polymorphism

Allows one interface to represent many forms.

Abstraction

Hides implementation details but does not specifically protect data access.

2 QUESTION 2



Which keyword allows a variable to belong to the class rather than an object?

a. final

b. private

✓ c. **static**

d. this

REASONING

A `static` variable belongs to the class itself, not to individual objects.

EXAMPLE

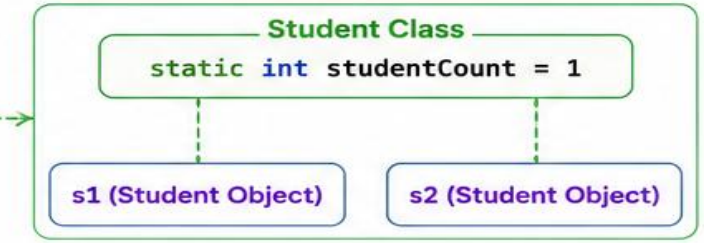
```
class Student {
    static int studentCount = 0;
}
```

→ `studentCount` belongs to the class `Student`

All `Student` objects share the same `studentCount`.

```
Student s1 = new Student();
Student s2 = new Student();

Student.studentCount++;
```



There is only one copy of `studentCount` regardless of how many objects exist.

WHY THE OTHERS ARE INCORRECT

a) final

Prevents modification.

```
final int x = 10;
x = 20;  : // Not allowed
```

b) private

Restricts access to the variable outside the class.

```
private int x = 10;
// Cannot be accessed directly
// outside the class
```

d) this

Refers to the current object.

```
this.name = name;
// Refers to the current object
```

How many objects can be created using the code below?

```
Student s1 = new Student();
Student s2 = new Student();
```

a. 0

b. 1

✓ c. 2

d. Unlimited

KEY TAKEAWAY

Using **new** creates a new object in memory every time.

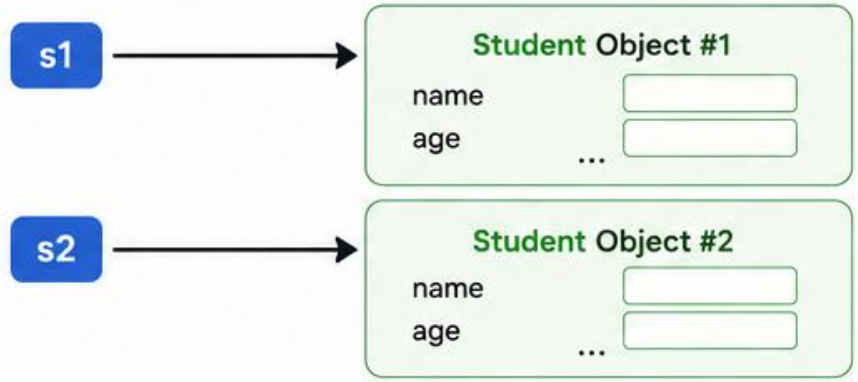
Singleton pattern is used when we want to restrict the creation of multiple objects and ensure only **one instance**.

REASONING

Each use of the **new** keyword creates a separate object.

```
Student s1 = new Student();   // Object 1
Student s2 = new Student();   // Object 2
```

MEMORY REPRESENTATION



Therefore, **2 objects** are created.

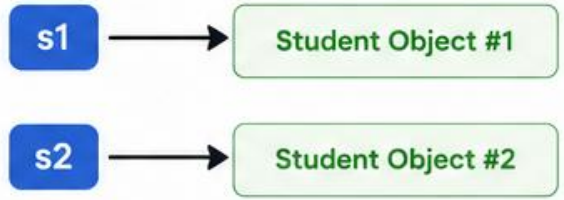
CONNECTION TO SINGLETON PATTERN

This question prepares students for Singleton.

Normally:

```
Student s1 = new Student();
Student s2 = new Student();
```

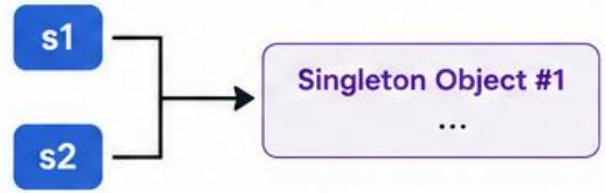
creates **2 objects**.



However, a **Singleton** class prevents this and ensures:

```
Singleton s1 = Singleton.getInstance();
Singleton s2 = Singleton.getInstance();
```

Both variables point to the **same object**.



Only **one object** is created and **shared** by all references.

DISCUSSION QUESTION



In Question 3, two objects were created.
Can you think of situations where creating multiple objects would be a problem?



EXPECTED RESPONSES

1 Database Connection Manager



Creating multiple connections can waste resources and may lead to inconsistent data access or connection conflicts.

2 Printer Queue



Multiple printer queue objects can cause duplicate print jobs, incorrect order, or communication issues with the printer.

3 ATM Controller



Multiple controllers can lead to incorrect balance updates, transaction conflicts, or security vulnerabilities.

4 Game Settings



Multiple settings objects can result in inconsistent configurations and a poor gaming experience.

5 Operating System Configuration



Multiple configuration objects can lead to conflicts, instability, and unpredictable system behavior.

These situations require only one object to ensure consistency, control, and efficient use of resources.
This is where the **Singleton Design Pattern** becomes useful!



CASE STUDY: UNIVERSITY PRINTING SYSTEM

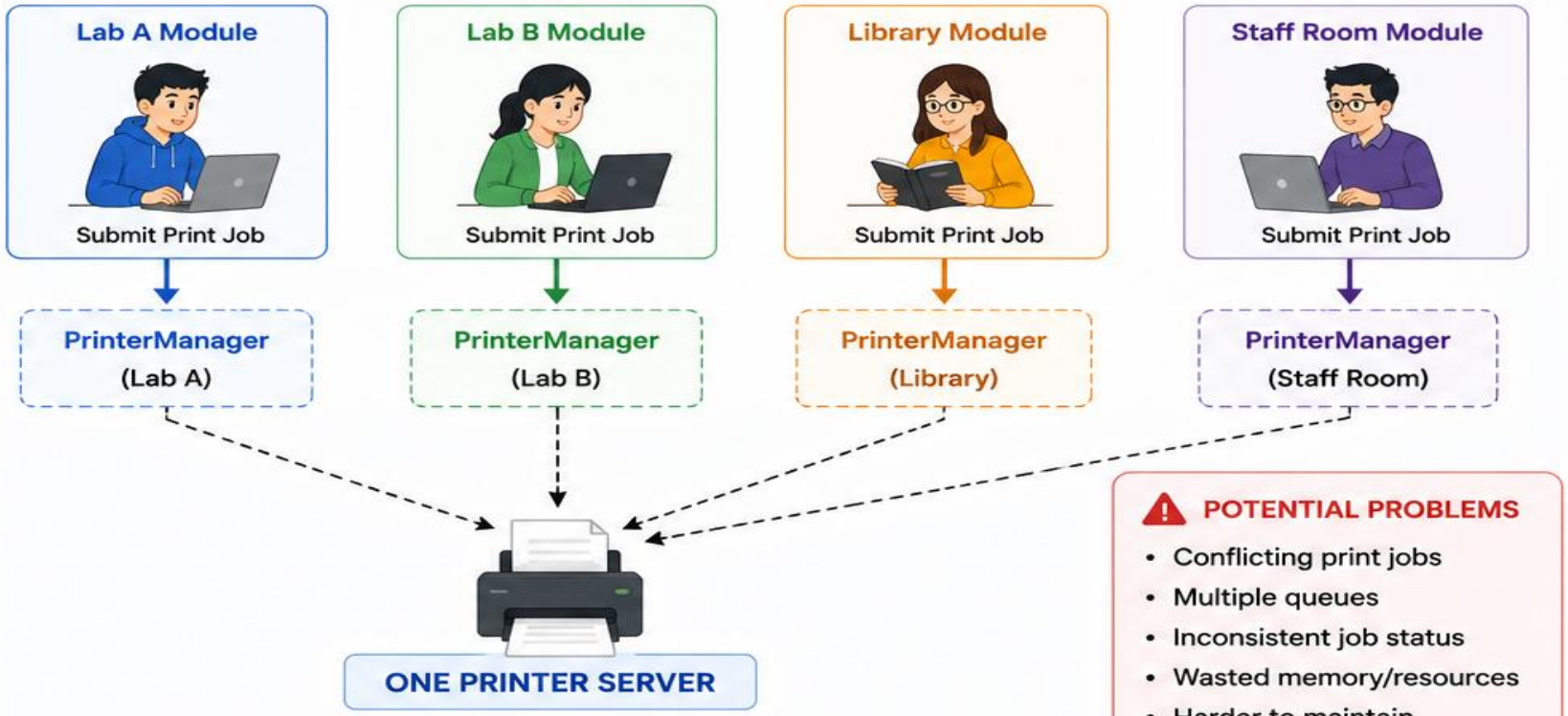
The university has **one** printer server.

Students submit printing jobs from:

- Lab A
- Lab B
- Library
- Staff Room

Each module creates its own **PrinterManager** object.

CURRENT SITUATION



POTENTIAL PROBLEMS

- Conflicting print jobs
- Multiple queues
- Inconsistent job status
- Wasted memory/resources
- Harder to maintain

DISCUSSION QUESTIONS

1. What problem might occur?
2. Should every module create its own PrinterManager?
3. How many PrinterManager objects should exist?

Think about it! A better solution will ensure only **one PrinterManager** manages all printing jobs.

This naturally leads us to the **Singleton Design Pattern.**

WHY SINGLETON?

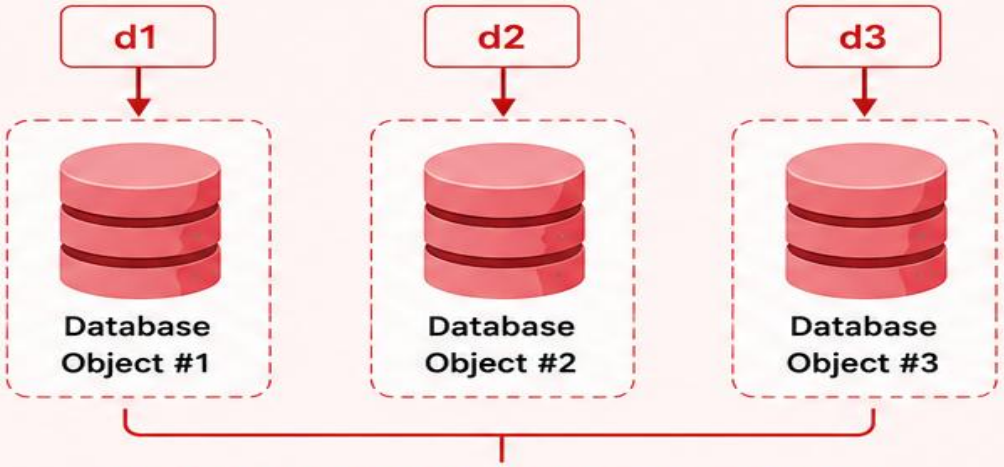
Some classes should have only one object in the entire system.



PROBLEM

Without restrictions:

```
Database d1 = new Database();
Database d2 = new Database();
Database d3 = new Database();
```



ISSUES

MEMORY WASTE

Each object consumes memory, leading to unnecessary duplication.

INCONSISTENT DATA

Different objects may have different data, causing inconsistent results.

RESOURCE CONFLICTS

Multiple objects may try to access the same resource (e.g., file, connection) at the same time, causing conflicts.

DIFFICULT SYSTEM MANAGEMENT

Tracking and managing multiple instances increases complexity and risk of bugs.

SOLUTION: SINGLETON PATTERN

Ensure that a class has **only one** object and provide a **global point of access** to it.



SINGLETON PATTERN DEFINITION

The Singleton Pattern restricts a class to have only one object and provides a global access point to it.

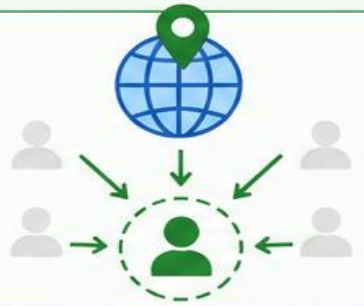
DEFINITION

A Singleton Pattern ensures:

- Only one instance of a class exists



- A global access point to that instance



REAL WORLD EXAMPLES



Principal of a school

A school can have only one principal at a time. All teachers and staff access the same principal.



National ID Database

There should be only one instance of the database managing all national ID records to ensure data consistency and security.



Air Traffic Control System

Only one control system manages all flights in a given airspace to avoid conflicts and ensure safety.

In short:

The Singleton Pattern ensures a class has one and only one instance and provides a global access point to it.

HOW IT WORKS

Step-by-step flow of the Singleton Pattern

1 STEP 1

Object does not exist.

```
instance = null;
```

Memory

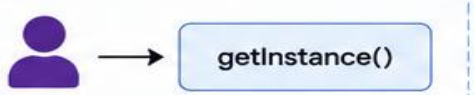
No object created yet.

The instance variable is null.

2 STEP 2

User calls:

```
getInstance();
```



User requests the single instance of the class.

3 STEP 3

Object is created.

```
if (instance == null) {
    instance = new Singleton();
}
```

Memory

Singleton Object

instance

Since instance is null, a new object is created and assigned to instance.

4 STEP 4

Same object is returned for future requests.

```
return instance;
```

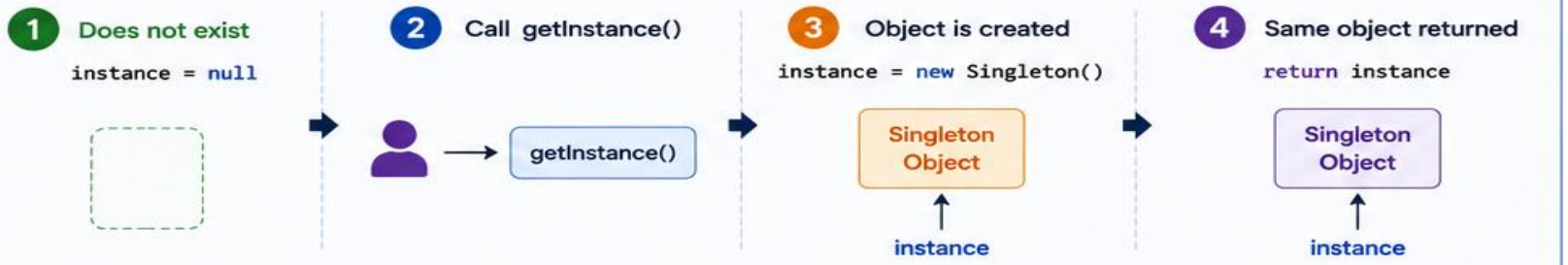
Memory

Singleton Object

instance

The same object is returned every time. No new object is created.

VISUAL SUMMARY



KEY POINT

No matter how many times **getInstance()** is called, the **same object** is returned. **Only one instance** exists in the entire application.

CODE WALKTHROUGH

Understanding the Singleton Pattern

1 Why is the constructor private?

- Prevents direct object creation
- Stops users from calling:

```
new Singleton();
```

- Ensures only the class itself can create the object

Singleton.java

```
1   public class Singleton {
2
3       private static Singleton instance;
4
5       private Singleton() {
6       }
7
8       public static Singleton getInstance() {
9           if (instance == null) {
10              instance = new Singleton();
11          }
12          return instance;
13      }
14  }
```

2 Why is instance static?

- Belongs to the class, not an object
- Shared by all users
- Only one copy exists in memory

3 What happens when getInstance() is called twice?

First call:

```
Singleton s1 = Singleton.getInstance();
```

- Object is created

Second call:

```
Singleton s2 = Singleton.getInstance();
```

- Existing object is returned
- No new object is created

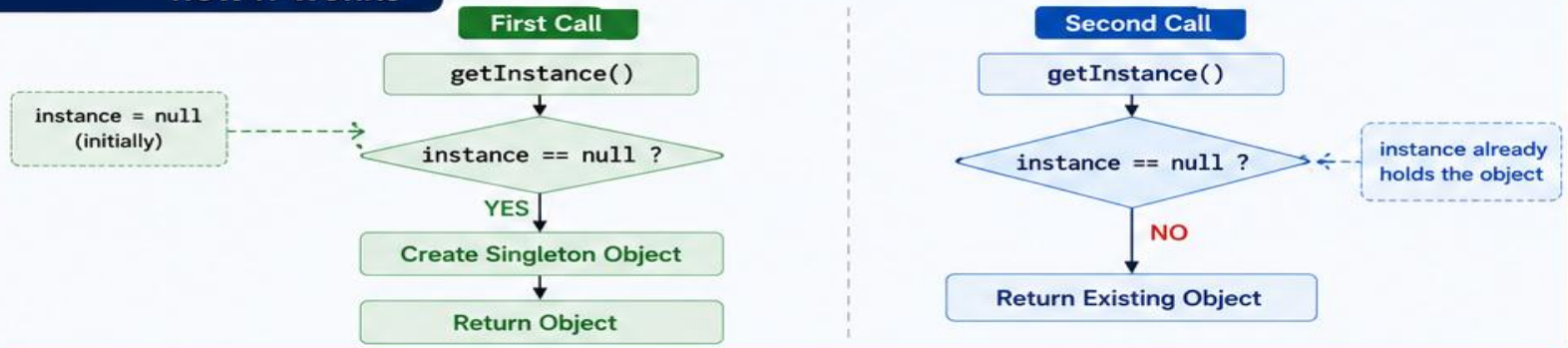
Result:

```
s1 == s2
```

returns

```
true
```

HOW IT WORKS



DISCUSS WITH YOUR PARTNER:

1. What would happen if the constructor were public?
2. What would happen if `instance` were not static?
3. Why does Singleton save memory?

TRACE THE EXECUTION

Predict the Output

CODE

```
Singleton s1 = Singleton.getInstance();
Singleton s2 = Singleton.getInstance();

System.out.println(s1 == s2);
```

SINGLETON CLASS (Simplified)

```
1  public class Singleton {
2      private static Singleton instance;
3
4      private Singleton() { }
5
6      public static Singleton getInstance() {
7          if (instance == null) {
8              instance = new Singleton();
9          }
10         return instance;
11     }
12 }
```

STEP-BY-STEP TRACE

Step	Statement	What Happens	instance	s1	s2	Diagram (Memory)
1	`Singleton s1 = Singleton.getInstance();`	`instance` is `null`, so a new Singleton object is created and assigned to `instance`. That object is returned and stored in s1.	`ref →`	`ref →`	—	instance → Singleton Object #1 s1 → Singleton Object #1
2	`Singleton s2 = Singleton.getInstance();`	`instance` is `not null`, so the existing object is returned and stored in s2.	`ref →`	`ref →`	`ref →`	instance → Singleton Object #1 s1 → Singleton Object #1 s2 → Singleton Object #1
3	`System.out.println (s1 == s2);`	`s1` and `s2` reference the same object. So the comparison is `true`.	`ref →`	`ref →`	`ref →`	instance → Singleton Object #1 s1 → Singleton Object #1 s2 → Singleton Object #1 Same Object

OUTPUT: `true`

DISCUSSION

Will it print:

- **a** `true`
- **b** `false`

Think:
Do s1 and s2 point to the same object or different objects?

TRACE THE EXECUTION

Predict the Output

CODE

```
Singleton s1 = Singleton.getInstance();
Singleton s2 = Singleton.getInstance();

System.out.println(s1 == s2);
```

ANSWER

✅ **a) true**

Both s1 and s2 refer to the same Singleton object.

So, s1 == s2 is true.

Output: true

SINGLETON CLASS (Simplified)

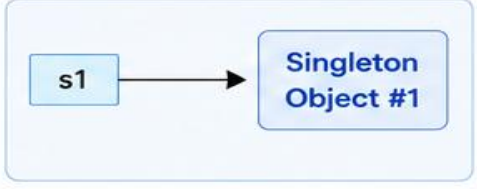
```
1   public class Singleton {
2       private static Singleton instance;
3
4       private Singleton() { }
5
6       public static Singleton getInstance() {
7           if (instance == null) {
8               instance = new Singleton();
9           }
10          return instance;
11      }
12  }
```

STEP-BY-STEP TRACE

1 First call

```
Singleton s1 =
Singleton.getInstance();
```

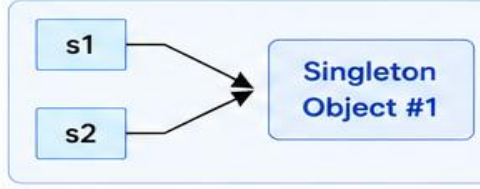
- instance is initially null
- A new Singleton object is created
- s1 points to that object



2 Second call

```
Singleton s2 =
Singleton.getInstance();
```

- **instance** is not null
- No new object is created
- The existing object is returned
- s2 points to the same object as s1



3 Comparison

```
System.out.println(s1 == s2);
```

The == operator compares whether two references point to the same object in memory.

Since both s1 and s2 point to Singleton Object #1:

```
s1 == s2
```

evaluates to:

true

KEY TAKEAWAY

The Singleton pattern ensures that only **one instance** of a class exists and provides a global access point to that instance.

? DISCUSSION QUESTION FOR STUDENTS

If the Singleton constructor was made public, would the following still print true?

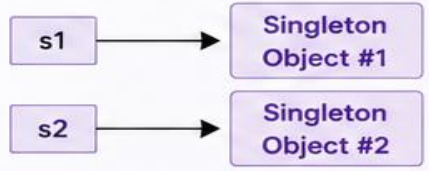
```
Singleton s1 = new Singleton();
Singleton s2 = new Singleton();

System.out.println(s1 == s2);
```

❌ **Answer:**

No, it would print false.

Because new creates two different objects in memory.



Applications of Singleton Pattern

Real-world examples where only one instance makes sense

1 Printer Queue



Manages all print jobs through a single queue to avoid conflicts and redundancy.

2 Database Connection Manager



Controls a single database connection pool for efficient resource usage and consistency.

3 Game Settings



Maintains a single set of game settings that is accessible across the entire game.

4 Software Drivers



Ensures only one instance of a driver is loaded to manage the hardware properly.

5 Building Automation



Centralizes control and monitoring of building systems through a single instance.

6 Payroll Processing



Guarantees that payroll is calculated and processed by only one instance to prevent errors and duplication.

7 Day-End Processing



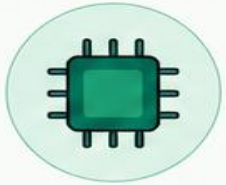
Ensures that end-of-day operations are executed once and only once across the system.

Key Takeaway: Use Singleton when exactly one instance is needed to coordinate actions, manage resources, or maintain consistency across the application.

Advantages & Disadvantages of Singleton

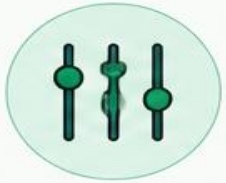
Every design choice has trade-offs.

ADVANTAGES



1. Saves memory

Only one instance is created and shared across the application, reducing memory usage.



2. Centralized control

Provides a single point of control over shared resources or configurations.



3. Easy access

Provides a sibbal access point to the instance through a simple method.



4. Prevents duplicate objects

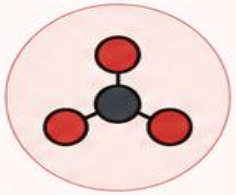
Ensures that only one instance exists, preventing conflicts and inconsistencies.

DISADVANTAGES



1. Difficult to test

Singletons introduce tight coupling, making unit testing harder because dependencies cannot be easily mocked.



2. Hidden dependencies

The global access makes it hard to see where and how the Singleton is being used, increasing coupling.



3. Can become a global variable

Overuse can lead to code that is hard to maintain and understand, similar to using global variables.



Key Takeaway: Use Singleton when you need exactly one instance to coordinate actions or manage resources, but **be cautious of overuse** as it can make your code harder to test, understand, and maintain.

EXTENDING SINGLETON FOR AN EXACT NUMBER OF OBJECTS

Use **Multiton Pattern** – allows a fixed number of instances instead of only one.

1. BASIC IDEA

Instead of maintaining a single instance, we maintain a collection of instances and control how many can be created.

Singleton	Multiton
Number of Objects = 1	Number of Objects = N
	

2. EXAMPLE: ALLOW EXACTLY 3 OBJECTS

```
import java.util.ArrayList;
import java.util.List;

public class PrinterManager {
    private static final int MAX_INSTANCES = 3;
    private static List<PrinterManager> instances =
            new ArrayList<>();

    private PrinterManager() { }

    public static PrinterManager getInstance() {
        if(instances.size() < MAX_INSTANCES) {
            instances.add(new PrinterManager());
        }
        return instances.get(instances.size() - 1);
    }
}
```

3. BETTER IMPLEMENTATION (ROUND ROBIN)

Suppose a company has 3 printers.

```
import java.util.ArrayList;
import java.util.List;

public class PrinterManager {
   private static final int MAX_PRINTERS = 3;
   private static List<PrinterManager> printers =
           new ArrayList<>();

   private static int nextPrinter = 0;

   private PrinterManager() { }

   public static PrinterManager getPrinter() {
      if(printers.size() < MAX_PRINTERS) {
          printers.add(new PrinterManager());
      }
      PrinterManager printer = printers.get(nextPrinter);
      nextPrinter = (nextPrinter + 1) % MAX_PRINTERS;
      return printer;
   }
}
```

USAGE

```
PrinterManager p1 = PrinterManager.getPrinter();
PrinterManager p2 = PrinterManager.getPrinter();
PrinterManager p3 = PrinterManager.getPrinter();
PrinterManager p4 = PrinterManager.getPrinter();
```

OUTPUT (ROUND ROBIN)

Call	Returned Object	Notes
p1	Printer 1	First object created
p2	Printer 2	Second object created
p3	Printer 3	Third object created
p4	Printer 1	Cycles back to first



4. GENERALIZED DESIGN

Requirement	Pattern	Number of Objects	Example
One database manager	Singleton	1	Single DB Connection
Three printers	Multiton	N (fixed)	3 Printers
Five ATM controllers	Multiton	N (fixed)	5 ATM Controllers
Ten worker threads	Object Pool	N (fixed/reusable)	10 Threads (reuse)
Unlimited customers	Normal Class	Unlimited	Any number of customers

5. CLASS DIAGRAM (MULTITON)



Responsibilities:

- Maintain a collection of instances
- Ensure no more than N objects are created
- Provide access to one of the available objects

6. TEACHING IDEA



After teaching Singleton, give students this challenge:

> **"The university has 4 printers. We don't want unlimited PrinterManager objects, but one Singleton is also inssfficient. How would you modify Singleton to allow exactly 4 objects?"**

This naturally introduces:

1. Singleton (1 object)
2. Multiton (N objects)
3. Object Pool (reuse limited objects)

and helps students understand that design patterns are solutions that can be adapted, not rigid rules.

KEY TAKEAWAY



- Singleton restricts the number of objects to 1.
- Multiton (a generalized Singleton) restricts the number to N.
- Object Pool extends this idea by reusing a fixed number of objects.
- Choose the pattern based on the number of instances your application needs.

Mastering the Singleton Design Pattern: One Instance, Global Access

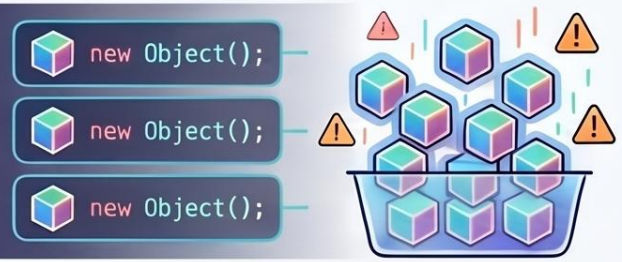
The Problem: Managing Multiple Objects

Resource and Data Conflicts



Multiple objects accessing one resource lead to inconsistent results and transaction conflicts.

Memory and Performance Waste



Every unnecessary object consumes memory, leading to inefficient duplication and difficult system management.

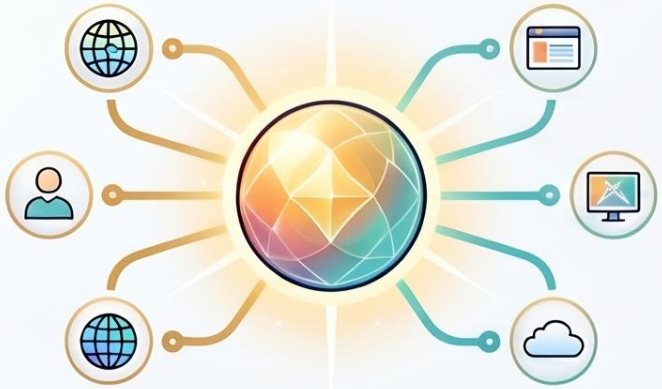
The University Printing Dilemma



Multiple printer managers for one server cause duplicate jobs and inconsistent status.

The Solution: The Singleton Pattern

Core Definition



Ensures a class has only one object while providing a universal access point.

Restricted Creation Logic



Uses a private constructor to prevent direct object creation via the 'new' keyword.

Real-World Implementations

Essential for database connection managers, air traffic control, and game settings.



Comparing Instance-Limiting Patterns

Requirement	Design Pattern	Number of Objects
Single Connection	Singleton	Exactly 1
Fixed Printers	Multiton	Fixed (N)
Worker Threads	Object Pool	Fixed/Reusable (N)

NotebookLM